ESPE UNIVERSIDAD DE LAS FUERZAS ARMADAS INNOVACIÓN PARA LA EXCELENCIA	INFORME DE PRÁCTICA DE LABORATORIO

DEPARTAMENTO:	**CIENCIAS DE LA COMPUTACIÓN**	**CARRERA:**	☒ **Ing. Software**		
ASIGNATURA:	ESTRUCTURA DE DATOS	**NIVEL:**	III	**FECHA:**	
DOCENTE:	Ing. Edgar Montaluisa, Mgtr.	**PRÁCTICA N°:**		**CALIFICACIÓN:**	

Título o tema de la práctica con una extensión máxima de 20 palabras. Tamaño de letra (TL) 14

Nombre del estudiante (es) TL 12

REQUISITOS FUNCIONALES

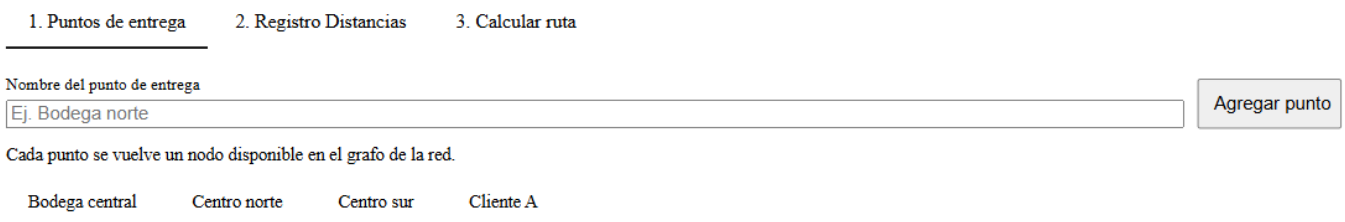
1. El usuario debe poder ingresar el nombre de los nodos.
2. El usuario debe poder ver los nodos agregados en una lista
3. El usuario debe poder registrar la distancia entre nodos.
4. El usuario debe poder seleccionar un nodo origen y un nodo destino para buscar una ruta.
5. El usuario debe poder ver la mejor ruta calcula entre nodos seleccionados
6. Integración debe poder ver una lista de las conexiones entre nodos

REQUISITOS NO FUNCIONALES

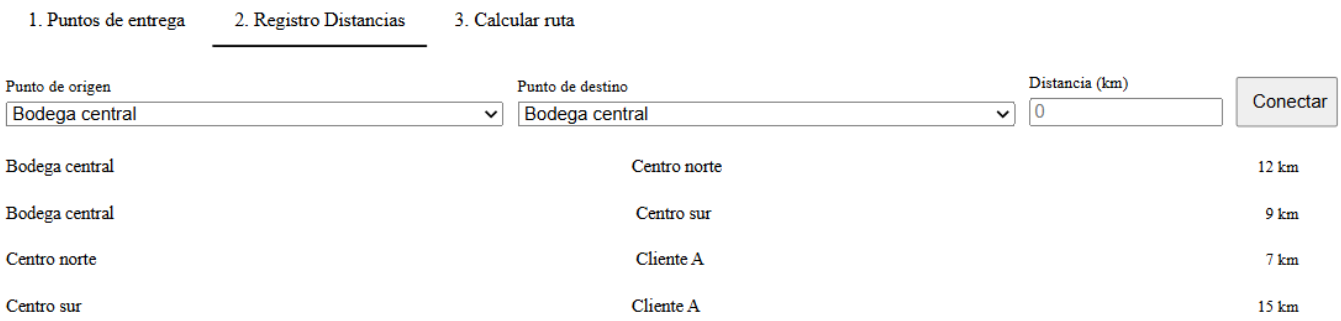
1. La página debe cargar rápido.
2. Debe tener diseño moderno.
3. Debe ser segura.
4. Debe ser fácil de usar

MOCKUPS:

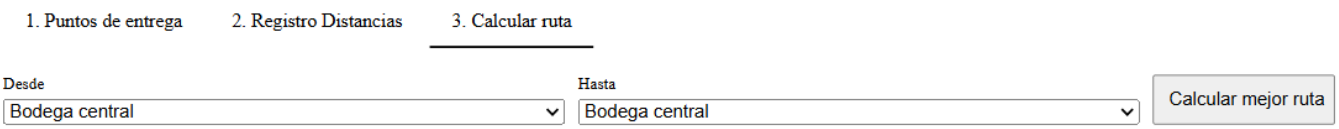
Ingreso de Nodos



Ingreso de Distancia



Cálculo de Ruta



CODIGO:

HTML

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Red de Puntos de Entrega</title>
  <link rel="preconnect" href="https://fonts.googleapis.com" />
  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin />
  <link href="https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700&display=swap" rel="stylesheet" />
  <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/@tabler/icons-webfont@latest/dist/tabler-icons.min.css" />
  <link rel="stylesheet" href="styles.css" />
</head>
<body>

  <header class="header">
    <div class="header-badge">
```

```
    <i class="ti ti-route"></i>
    Análisis de rutas
  </div>
  <h1>Red de Puntos de Entrega</h1>
  <p>Registra puntos, conecta ubicaciones con distancias y calcula la mejor ruta entre dos puntos.</p>
</header>

<main class="app-card">

  <!-- Tabs -->
  <nav class="app-tabs" role="tablist" aria-label="Secciones de la aplicación">
    <button class="app-tab active" role="tab" data-tab="nodos"      onclick="switchTab('nodos')"      aria-selected="true">
      <i class="ti ti-map-pin" aria-hidden="true"></i>Puntos de entrega
    </button>
    <button class="app-tab"       role="tab" data-tab="conexiones" onclick="switchTab('conexiones')" aria-selected="false">
      <i class="ti ti-git-branch" aria-hidden="true"></i>Distancias
    </button>
    <button class="app-tab"       role="tab" data-tab="ruta"       onclick="switchTab('ruta')"       aria-selected="false">
      <i class="ti ti-route-2" aria-hidden="true"></i>Calcular ruta
    </button>
  </nav>

  <!-- Panel 1: Nodos -->
  <section class="panel active" id="panel-nodos" role="tabpanel" aria-label="Puntos de entrega">
    <p class="panel-title">Agregar punto de entrega</p>
    <p class="panel-desc">Cada punto representa un nodo en la red logística. Los puntos eliminados también borran sus conexiones.</p>

    <div class="field-row">
      <div class="field">
        <label for="node-name">Nombre del punto</label>
        <input type="text" id="node-name" placeholder="Ej. Bodega norte" autocomplete="off" />
```

```
    </div>
    <button class="btn btn-primary" onclick="addNode()">
      <i class="ti ti-plus" aria-hidden="true"></i>Agregar
    </button>
  </div>

  <p id="node-error" role="alert" style="font-size:12px; color:var(--danger); margin-bottom:10px; display:none;"></p>

  <div class="divider"></div>
  <div id="node-empty" class="node-empty" style="display:none;">
    <i class="ti ti-map-pin-off" aria-hidden="true"></i>
    Todavía no hay puntos de entrega registrados.
  </div>
  <div id="node-list" aria-live="polite" aria-label="Lista de puntos de entrega"></div>
</section>

<!-- Panel 2: Conexiones -->
<section class="panel" id="panel-conexiones" role="tabpanel" aria-label="Registro de distancias">
  <p class="panel-title">Registrar conexión</p>
  <p class="panel-desc">Indica el origen, el destino y la distancia en kilómetros. Las conexiones son bidireccionales.</p>

  <div class="field-row">
    <div class="field">
      <label for="conn-from">Punto de origen</label>
      <select id="conn-from"></select>
    </div>
    <div class="field">
      <label for="conn-to">Punto de destino</label>
      <select id="conn-to"></select>
    </div>
    <div class="field" style="max-width:120px;">
```

```
        <label for="conn-km">Distancia (km)</label>
        <input type="number" id="conn-km" min="0.1" step="0.1" placeholder="0.0" />
      </div>
      <button class="btn btn-primary" onclick="addConnection()">
        <i class="ti ti-link" aria-hidden="true"></i>Conectar
      </button>
    </div>

    <p id="conn-error" role="alert" style="font-size:12px; color:var(--danger); margin-bottom:10px; display:none;"></p>

    <div class="divider"></div>

    <div id="conn-list" aria-live="polite" aria-label="Lista de conexiones"></div>
    <div id="conn-empty" class="conn-empty" style="display:none;">
      <i class="ti ti-git-merge" aria-hidden="true"></i>
      Todavía no hay conexiones registradas.
    </div>
  </section>

  <!-- Panel 3: Ruta -->
  <section class="panel" id="panel-ruta" role="tabpanel" aria-label="Calcular ruta">
    <p class="panel-title">Calcular mejor ruta</p>
    <p class="panel-desc">El algoritmo evalúa la conexión directa y rutas con un nodo intermedio, eligiendo la de menor distancia total.</p>

    <div class="field-row">
      <div class="field">
        <label for="ruta-from">Desde</label>
        <select id="ruta-from"></select>
      </div>
      <div class="field">
```

```
        <label for="ruta-to">Hasta</label>
        <select id="ruta-to"></select>
      </div>
      <button class="btn btn-primary" onclick="calcularRuta()">
        <i class="ti ti-map-2" aria-hidden="true"></i>Calcular
      </button>
    </div>

    <div id="ruta-result" aria-live="polite" aria-label="Resultado de la ruta"></div>
  </section>

</main>

<script src="app.js"></script>
</body>
</html>
```

CSS:

```
*, *::before, *::after { box-sizing: border-box; margin: 0; padding: 0; }

:root {
  --bg: #0f1117;
  --bg-card: #1a1d27;
  --bg-elevated: #222536;
  --bg-input: #13151f;
  --border: #2e3145;
  --border-hover: #4a4f6b;
  --accent: #6c63ff;
  --accent-hover: #8078ff;
  --accent-dim: rgba(108, 99, 255, 0.15);
  --success: #22d3a0;
  --success-dim: rgba(34, 211, 160, 0.12);
```

```
  --danger: #ff5f7e;
  --danger-dim: rgba(255, 95, 126, 0.12);
  --text-primary: #eef0f8;
  --text-secondary: #8b90b0;
  --text-muted: #50556e;
  --radius-sm: 6px;
  --radius-md: 10px;
  --radius-lg: 14px;
  --shadow-card: 0 4px 24px rgba(0,0,0,0.4);
  --shadow-btn: 0 2px 12px rgba(108,99,255,0.35);
  --transition: 0.18s ease;
}

body {
  background: var(--bg);
  color: var(--text-primary);
  font-family: 'Inter', system-ui, -apple-system, sans-serif;
  min-height: 100vh;
  display: flex;
  flex-direction: column;
  align-items: center;
  padding: 40px 16px 80px;
}

/* ── Header ── */
.header {
  text-align: center;
  margin-bottom: 40px;
}
.header-badge {
  display: inline-flex;
```

```
  align-items: center;
  gap: 6px;
  background: var(--accent-dim);
  border: 1px solid rgba(108,99,255,0.3);
  color: var(--accent);
  font-size: 11px;
  font-weight: 600;
  letter-spacing: 0.08em;
  text-transform: uppercase;
  padding: 4px 12px;
  border-radius: 100px;
  margin-bottom: 16px;
}
.header h1 {
  font-size: 2rem;
  font-weight: 700;
  letter-spacing: -0.02em;
  background: linear-gradient(135deg, #eef0f8 0%, #8b90b0 100%);
  -webkit-background-clip: text;
  -webkit-text-fill-color: transparent;
  background-clip: text;
  margin-bottom: 8px;
}
.header p {
  font-size: 14px;
  color: var(--text-secondary);
  max-width: 420px;
  margin: 0 auto;
  line-height: 1.6;
}
```

```
/* ── Main card ── */
.app-card {
  width: 100%;
  max-width: 780px;
  background: var(--bg-card);
  border: 1px solid var(--border);
  border-radius: var(--radius-lg);
  box-shadow: var(--shadow-card);
  overflow: hidden;
}

/* ── Tabs ── */
.app-tabs {
  display: flex;
  background: var(--bg);
  border-bottom: 1px solid var(--border);
  padding: 0 4px;
}
.app-tab {
  display: flex;
  align-items: center;
  gap: 7px;
  padding: 14px 18px;
  font-size: 13px;
  font-weight: 500;
  color: var(--text-muted);
  cursor: pointer;
  border-bottom: 2px solid transparent;
  margin-bottom: -1px;
  transition: color var(--transition);
  white-space: nowrap;
```

```
  user-select: none;
}
.app-tab i { font-size: 15px; }
.app-tab:hover { color: var(--text-secondary); }
.app-tab.active {
  color: var(--accent);
  border-bottom-color: var(--accent);
}

/* ── Panels ── */
.panel { display: none; padding: 28px; }
.panel.active { display: block; }

.panel-title {
  font-size: 15px;
  font-weight: 600;
  color: var(--text-primary);
  margin-bottom: 4px;
}
.panel-desc {
  font-size: 13px;
  color: var(--text-secondary);
  margin-bottom: 20px;
  line-height: 1.5;
}

/* ── Form elements ── */
.field-row {
  display: flex;
  gap: 10px;
  align-items: flex-end;
```

```
  flex-wrap: wrap;
  margin-bottom: 16px;
}
.field {
  display: flex;
  flex-direction: column;
  gap: 6px;
  flex: 1;
  min-width: 130px;
}
.field label {
  font-size: 12px;
  font-weight: 500;
  color: var(--text-secondary);
  letter-spacing: 0.02em;
}

input[type="text"],
input[type="number"],
select {
  background: var(--bg-input);
  border: 1px solid var(--border);
  border-radius: var(--radius-sm);
  color: var(--text-primary);
  font-size: 13px;
  font-family: inherit;
  height: 38px;
  padding: 0 12px;
  outline: none;
  transition: border-color var(--transition), box-shadow var(--transition);
  width: 100%;
```

```
  appearance: none;
  -webkit-appearance: none;
}
input[type="text"]:focus,
input[type="number"]:focus,
select:focus {
  border-color: var(--accent);
  box-shadow: 0 0 0 3px var(--accent-dim);
}
input::placeholder { color: var(--text-muted); }

select {
  background-image: url("data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg' width='12' height='12' viewBox='0 0 24 24' fill='none' stroke='%238b90b0' stroke-width='2'%3E%3Cpath d='M6 9l6 6 6-6'/%3E%3C/svg%3E");
  background-repeat: no-repeat;
  background-position: right 10px center;
  padding-right: 32px;
}
select option { background: var(--bg-card); }

/* ── Buttons ── */
.btn {
  display: inline-flex;
  align-items: center;
  gap: 6px;
  height: 38px;
  padding: 0 18px;
  border: none;
  border-radius: var(--radius-sm);
  font-size: 13px;
  font-weight: 600;
```

```
  font-family: inherit;
  cursor: pointer;
  transition: all var(--transition);
  white-space: nowrap;
  flex-shrink: 0;
}
.btn i { font-size: 15px; }

.btn-primary {
  background: var(--accent);
  color: #fff;
  box-shadow: var(--shadow-btn);
}
.btn-primary:hover {
  background: var(--accent-hover);
  transform: translateY(-1px);
  box-shadow: 0 4px 16px rgba(108,99,255,0.45);
}
.btn-primary:active { transform: translateY(0); }

.btn-ghost {
  background: transparent;
  border: 1px solid var(--border);
  color: var(--text-secondary);
}
.btn-ghost:hover {
  border-color: var(--border-hover);
  color: var(--text-primary);
  background: var(--bg-elevated);
}
```

```
/* ── Node list ── */
.node-empty {
  font-size: 13px;
  color: var(--text-muted);
  text-align: center;
  padding: 24px 0;
  display: flex;
  flex-direction: column;
  align-items: center;
  gap: 8px;
}
.node-empty i { font-size: 28px; opacity: 0.4; }

.node-ol {
  list-style: none;
  margin: 0;
  padding: 0;
}
.node-item {
  display: grid;
  grid-template-columns: 28px 18px 1fr auto;
  align-items: center;
  gap: 10px;
  padding: 10px 14px;
  border-radius: var(--radius-sm);
  font-size: 13px;
  transition: background var(--transition);
}
.node-item:hover { background: var(--bg-elevated); }
.node-item + .node-item { border-top: 1px solid var(--border); }
.node-num {
```

```
  font-size: 12px;
  font-weight: 600;
  color: var(--text-muted);
  text-align: right;
}
.node-icon { font-size: 14px; color: var(--accent); }
.node-name { font-weight: 500; color: var(--text-primary); }
.node-del {
  display: flex;
  align-items: center;
  background: none;
  border: none;
  cursor: pointer;
  color: var(--text-muted);
  padding: 4px;
  border-radius: var(--radius-sm);
  transition: color var(--transition), background var(--transition);
}
.node-del:hover { color: var(--danger); background: var(--danger-dim); }
.node-del i { font-size: 15px; }

/* ── Connection list ── */
.conn-list { margin-top: 4px; }
.conn-empty {
  font-size: 13px;
  color: var(--text-muted);
  text-align: center;
  padding: 24px 0;
  display: flex;
  flex-direction: column;
  align-items: center;
```

```
  gap: 8px;
}
.conn-empty i { font-size: 28px; opacity: 0.4; }

.conn-row {
  display: grid;
  grid-template-columns: 1fr auto 1fr auto auto;
  align-items: center;
  gap: 10px;
  padding: 10px 14px;
  border-radius: var(--radius-sm);
  font-size: 13px;
  transition: background var(--transition);
}
.conn-row:hover { background: var(--bg-elevated); }
.conn-row + .conn-row { border-top: 1px solid var(--border); }
.conn-from { font-weight: 500; color: var(--text-primary); }
.conn-to { font-weight: 500; color: var(--text-primary); }
.conn-arrow { color: var(--text-muted); display: flex; align-items: center; }
.conn-arrow i { font-size: 14px; }
.badge-km {
  background: var(--accent-dim);
  color: var(--accent);
  font-size: 11px;
  font-weight: 600;
  padding: 3px 10px;
  border-radius: 100px;
  white-space: nowrap;
  letter-spacing: 0.03em;
}
.conn-del {
```

```
  display: flex;
  align-items: center;
  background: none;
  border: none;
  cursor: pointer;
  color: var(--text-muted);
  padding: 4px;
  border-radius: var(--radius-sm);
  transition: color var(--transition), background var(--transition);
}
.conn-del:hover { color: var(--danger); background: var(--danger-dim); }
.conn-del i { font-size: 15px; }

/* ── Route result ── */
.results-list {
  display: flex;
  flex-direction: column;
  gap: 10px;
  margin-top: 16px;
}

.result-card {
  background: var(--bg-elevated);
  border: 1px solid var(--border);
  border-radius: var(--radius-md);
  padding: 20px 24px;
}
.result-card.best { border-color: rgba(108,99,255,0.45); }
.result-card.alt  { opacity: 0.72; }

.result-meta {
```

```
  display: flex;
  align-items: center;
  gap: 10px;
  margin-bottom: 12px;
}
.result-label {
  font-size: 11px;
  font-weight: 600;
  letter-spacing: 0.08em;
  text-transform: uppercase;
}
.result-type {
  font-size: 12px;
  color: var(--text-muted);
}
.result-label.is-direct { color: var(--accent); }
.result-label.is-alt    { color: var(--text-secondary); }
.result-label.is-error  { color: var(--danger); }

.route-steps {
  display: flex;
  align-items: center;
  flex-wrap: wrap;
  gap: 8px;
  margin-bottom: 16px;
}
.route-node {
  background: var(--bg-card);
  border: 1px solid var(--border);
  border-radius: var(--radius-sm);
  padding: 6px 14px;
```

```
  font-size: 14px;
  font-weight: 600;
  color: var(--text-primary);
}
.route-node.highlight { border-color: var(--accent); color: var(--accent); }
.route-arrow-icon {
  color: var(--text-muted);
  display: flex;
  align-items: center;
}
.route-arrow-icon i { font-size: 16px; }

.result-distance {
  font-size: 36px;
  font-weight: 700;
  letter-spacing: -0.03em;
  color: var(--text-primary);
  line-height: 1;
}
.result-distance span {
  font-size: 16px;
  font-weight: 400;
  color: var(--text-secondary);
  margin-left: 4px;
}

.result-empty {
  font-size: 13px;
  color: var(--text-muted);
  text-align: center;
  padding: 20px 0;
```

```
}

/* ── Divider ── */
.divider {
  height: 1px;
  background: var(--border);
  margin: 20px 0;
}

/* ── Responsive ── */
@media (max-width: 560px) {
  body { padding: 20px 10px 60px; }
  .header h1 { font-size: 1.5rem; }
  .panel { padding: 20px 16px; }
  .app-tab { padding: 12px 12px; font-size: 12px; }
  .conn-row { grid-template-columns: 1fr auto 1fr auto; }
  .conn-row .badge-km { display: none; }
}
```

JS:

```
let nodes = [];
let connections = [];

// ── Tabs ──────────────────────────────────────────────────────────────────────

function switchTab(tab) {
  document.querySelectorAll('.app-tab').forEach(t =>
    t.classList.toggle('active', t.dataset.tab === tab)
  );
  document.querySelectorAll('.panel').forEach(p =>
    p.classList.toggle('active', p.id === 'panel-' + tab)
  );
```

```
}

// ── Nodes ──────────────────────────────────────────────────────────────────

function addNode() {
  const input = document.getElementById('node-name');
  const val = input.value.trim();
  if (!val) return showError('node-error', 'Ingresa un nombre para el punto de entrega.');
  if (nodes.includes(val)) return showError('node-error', `El punto "${val}" ya existe.`);
  clearError('node-error');
  nodes.push(val);
  input.value = '';
  renderNodes();
  renderSelects();
}

function removeNode(name) {
  nodes = nodes.filter(n => n !== name);
  connections = connections.filter(c => c.from !== name && c.to !== name);
  renderNodes();
  renderSelects();
  renderConnections();
}

function renderNodes() {
  const list  = document.getElementById('node-list');
  const empty = document.getElementById('node-empty');

  if (!nodes.length) {
    list.innerHTML = '';
    empty.style.display = 'flex';
```

```
    return;
  }

  empty.style.display = 'none';
  list.innerHTML = `<ol class="node-ol">
    ${nodes.map((n, i) => `
      <li class="node-item">
        <span class="node-num">${i + 1}</span>
        <i class="ti ti-map-pin-filled node-icon" aria-hidden="true"></i>
        <span class="node-name">${escapeHtml(n)}</span>
        <button class="node-del" onclick="removeNode(${JSON.stringify(n)})" aria-label="Eliminar ${escapeHtml(n)}">
          <i class="ti ti-trash"></i>
        </button>
      </li>
    `).join('')}
  </ol>`;
}

// ── Connections ──────────────────────────────────────────────────────────────
function addConnection() {
  const from = document.getElementById('conn-from').value;
  const to   = document.getElementById('conn-to').value;
  const km   = parseFloat(document.getElementById('conn-km').value);

  if (!from || !to) return showError('conn-error', 'Selecciona los puntos de origen y destino.');
  if (from === to)  return showError('conn-error', 'El origen y el destino no pueden ser el mismo punto.');
  if (!km || km <= 0) return showError('conn-error', 'Ingresa una distancia válida mayor a 0.');

  const duplicate = connections.find(c =>
    (c.from === from && c.to === to) || (c.from === to && c.to === from)
  );
```

```
  if (duplicate) return showError('conn-error', 'Ya existe una conexión entre estos dos puntos.');

  clearError('conn-error');
  connections.push({ from, to, km });
  document.getElementById('conn-km').value = '';
  renderConnections();
}

function removeConnection(idx) {
  connections.splice(idx, 1);
  renderConnections();
}

function renderConnections() {
  const list  = document.getElementById('conn-list');
  const empty = document.getElementById('conn-empty');

  if (!connections.length) {
    list.innerHTML = '';
    empty.style.display = 'flex';
    return;
  }

  empty.style.display = 'none';
  list.innerHTML = connections.map((c, i) => `
    <div class="conn-row">
      <span class="conn-from">${escapeHtml(c.from)}</span>
      <span class="conn-arrow"><i class="ti ti-arrow-right"></i></span>
      <span class="conn-to">${escapeHtml(c.to)}</span>
      <span class="badge-km">${c.km} km</span>
      <button class="conn-del" onclick="removeConnection(${i})" aria-label="Eliminar conexión">
```

```
      <i class="ti ti-trash"></i>
    </button>
  </div>
 `).join('');
}

// ── Route ──────────────────────────────────────────────────────────────────

function calcularRuta() {
 const from   = document.getElementById('ruta-from').value;
 const to     = document.getElementById('ruta-to').value;
 const result = document.getElementById('ruta-result');

 if (!from || !to) {
  result.innerHTML = emptyResult('Selecciona los puntos de origen y destino.');
  return;
 }
 if (from === to) {
  result.innerHTML = emptyResult('El origen y el destino deben ser puntos distintos.');
  return;
 }

 const paths = findAllPaths(from, to);

 if (!paths.length) {
  result.innerHTML = `
   <div class="result-card">
    <div class="result-label is-error">Sin ruta disponible</div>
    <p style="font-size:14px; color:var(--text-secondary);">No existe ninguna ruta entre <strong>${escapeHtml(from)}</strong> y
<strong>${escapeHtml(to)}</strong>.</p>
   </div>`;
  return;
```

```
  }

  const shown = paths.slice(0, 5);
  result.innerHTML = `<div class="results-list">${shown.map((p, i) => buildResult(p.route, p.km, i === 0, i + 1)).join('')}</div>`;
}

function findAllPaths(from, to) {
  const paths = [];

  function dfs(current, path, totalKm, visited) {
    if (current === to) {
      paths.push({ route: [...path], km: totalKm });
      return;
    }
    connections.forEach(c => {
      let neighbor = null, edgeKm = 0;
      if (c.from === current && !visited.has(c.to)) {
        neighbor = c.to; edgeKm = c.km;
      } else if (c.to === current && !visited.has(c.from)) {
        neighbor = c.from; edgeKm = c.km;
      }
      if (neighbor !== null) {
        visited.add(neighbor);
        path.push(neighbor);
        dfs(neighbor, path, totalKm + edgeKm, visited);
        path.pop();
        visited.delete(neighbor);
      }
    });
  }
```

```
  const visited = new Set([from]);
  dfs(from, [from], 0, visited);
  return paths.sort((a, b) => a.km - b.km);
}

function buildResult(route, total, isBest, rank) {
  const stops = route.length - 2;
  const typeLabel = stops === 0
    ? 'Ruta directa'
    : `${stops} parada${stops !== 1 ? 's' : ''} intermedia${stops !== 1 ? 's' : ''}`;
  const rankLabel  = isBest ? 'Mejor ruta' : `Alternativa ${rank}`;
  const cardClass  = isBest ? 'best' : 'alt';
  const labelClass = isBest ? 'is-direct' : 'is-alt';

  const steps = route.map((n, i) => {
    const isEndpoint = i === 0 || i === route.length - 1;
    const pill = `<span class="route-node ${isEndpoint ? 'highlight' : ''}">${escapeHtml(n)}</span>`;
    return i === 0 ? pill : `<span class="route-arrow-icon"><i class="ti ti-arrow-right"></i></span>${pill}`;
  }).join('');

  return `
    <div class="result-card ${cardClass}">
      <div class="result-meta">
        <span class="result-label ${labelClass}">${rankLabel}</span>
        <span class="result-type">${typeLabel}</span>
      </div>
      <div class="route-steps">${steps}</div>
      <div class="result-distance">${round1(total)}<span>km totales</span></div>
    </div>`;
}
```

```
function emptyResult(msg) {
  return `<p class="result-empty">${escapeHtml(msg)}</p>`;
}

// ── Selects ──────────────────────────────────────────────────────────────
function renderSelects() {
  ['conn-from', 'conn-to', 'ruta-from', 'ruta-to'].forEach(id => {
    const sel = document.getElementById(id);
    const prev = sel.value;
    sel.innerHTML = nodes.map(n => `<option value="${escapeHtml(n)}">${escapeHtml(n)}</option>`).join('');
    if (nodes.includes(prev)) sel.value = prev;
  });
}

// ── Utilities ────────────────────────────────────────────────────────────
function escapeHtml(str) {
  return str.replace(/&/g,'&amp;').replace(/</g,'&lt;').replace(/>/g,'&gt;').replace(/"/g,'&quot;');
}

function round1(n) { return Math.round(n * 10) / 10; }

function showError(id, msg) {
  const el = document.getElementById(id);
  if (el) { el.textContent = msg; el.style.display = 'block'; }
}

function clearError(id) {
  const el = document.getElementById(id);
  if (el) { el.textContent = ''; el.style.display = 'none'; }
}
```

```
// ── Init ──────────────────────────────────────────────────────────────────────

document.addEventListener('DOMContentLoaded', () => {
  renderNodes();
  renderSelects();
  renderConnections();

  document.getElementById('node-name').addEventListener('keydown', e => {
    if (e.key === 'Enter') addNode();
  });
});
```